

Reviewer Pack — Minimal Order of p-Adic L-Functions and Circuit Satisfiability...

Entry d8966ba43beb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 15:40:49 UTC

Minimal Order of p-Adic L-Functions and Circuit Satisfiability Threshold

Entry ID: d8966ba43beb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 15:40:49 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (L-Functions) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Satisfiability)

Statement:

For every Boolean circuit φ with n inputs, the minimal order of a p-adic L-function associated with φ is linearly correlated with its satisfiability threshold $k(\varphi)$, such that the minimal order of the L-function $\leq \alpha * \log(n) + \beta$ for some constants α and β .

Rationale (proposer's reasoning):

p-adic L-functions encode arithmetic properties of numbers, while circuit satisfiability threshold measures the difficulty of evaluating Boolean circuits. The conjecture suggests a potential bridge between these areas by proposing a quantitative relation between the order of p-adic L-functions and the complexity of circuits.

Taxonomy category: p_adic_analysis_circuit_satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7234ce83f96c5c6a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit φ with n inputs, if the minimal order of the associated p-adic L-function is $\leq \alpha * \log(n) + \beta$ for any constant pair (α, β) , or if there exists at least one seed where the minimal order exceeds $\alpha * \log(n) + \beta$ by more than 3 standard deviations from the mean, then the conjecture is supported or falsified, respectively.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_circuit(n, depth):
    if n == 1 or depth == 0:
        return [random.choice([0, 1])]

    inputs = []
    for _ in range(2 ** (n - 1)):
        sub_inputs = generate_circuit(n // 2, depth - 1)
        inputs.append(sub_inputs[0] & sub_inputs[1])

    return inputs

def satisfiability_threshold(circuit):
    n = len(circuit)
    if n == 1:
        return circuit[0]

    threshold = 0
    for i in range(n):
        if circuit[i]:
            threshold += 2 ** (n - i - 1)

    return threshold

def p_adic_l_function_order(n, k):
    # Simplified approximation of the order of a p-adic L-function
    return n * math.log(k, 2)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5): # Test with 5 different instances per size
            circuit = generate_circuit(n, random.randint(1, 3))
```

```

        k = satisfiability_threshold(circuit)
        order = p_adic_l_function_order(n, k)
        results.append((n, k, order))

n_max = max(n for n, _, _ in results)
if n_max < 16:
    return {
        "metric_name": "p-adic L-function order",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "n_max < 16"
    }

mean_order = sum(order for _, _, order in results) / len(results)
std_dev = math.sqrt(sum((order - mean_order) ** 2 for _, _, order in results) / len(results))

alpha, beta = random.random(), random.random()
expected_bound = [alpha * math.log(n, 2) + beta for n, _, _ in results]

conjecture_holds = all(abs(order - bound) <= 3 * std_dev for order, _, bound in zip(results, n_max, expected_bound))
counterexample = "" if conjecture_holds else "order > alpha * log(n) + beta by more than 3 std dev"

return {
    "metric_name": "p-adic L-function order",
    "metric_value": mean_order,
    "instances_tested": len(results),
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    mean_order = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_order) ** 2 for result in results if result["metric_value"] is not None) / len(results))

    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"order > alpha * log(n) + beta by more than 3 std dev\"")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99585bf0.py", line 90, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99585bf0.py", line 50, in run_trial
    circuit = generate_circuit(n, random.randint(1, 3))
             ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99585bf0.py", line 24, in generate_circuit
    sub_inputs = generate_circuit(n // 2, depth - 1)
                ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_99585bf0.py", line 25, in generate_circuit
    inputs.append(sub_inputs[0] & sub_inputs[1])
                   ~~~~~^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide evidence for or against the conjecture. | next: Re-run the test with increased robustness checks and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13370
2	preregistration	ollama_remote	glm4:latest	0	0	9787
3	novelty	ollama_remote	glm4:latest	0	0	13000
4	novelty	ollama_remote	glm4:latest	0	0	8778
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16849
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13694
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12396
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11644
9	judge	ollama_remote	glm4:latest	0	0	13379

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112898 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d8966ba43beb.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d8966ba43beb.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d8966ba43beb.tar.gz (if generated)